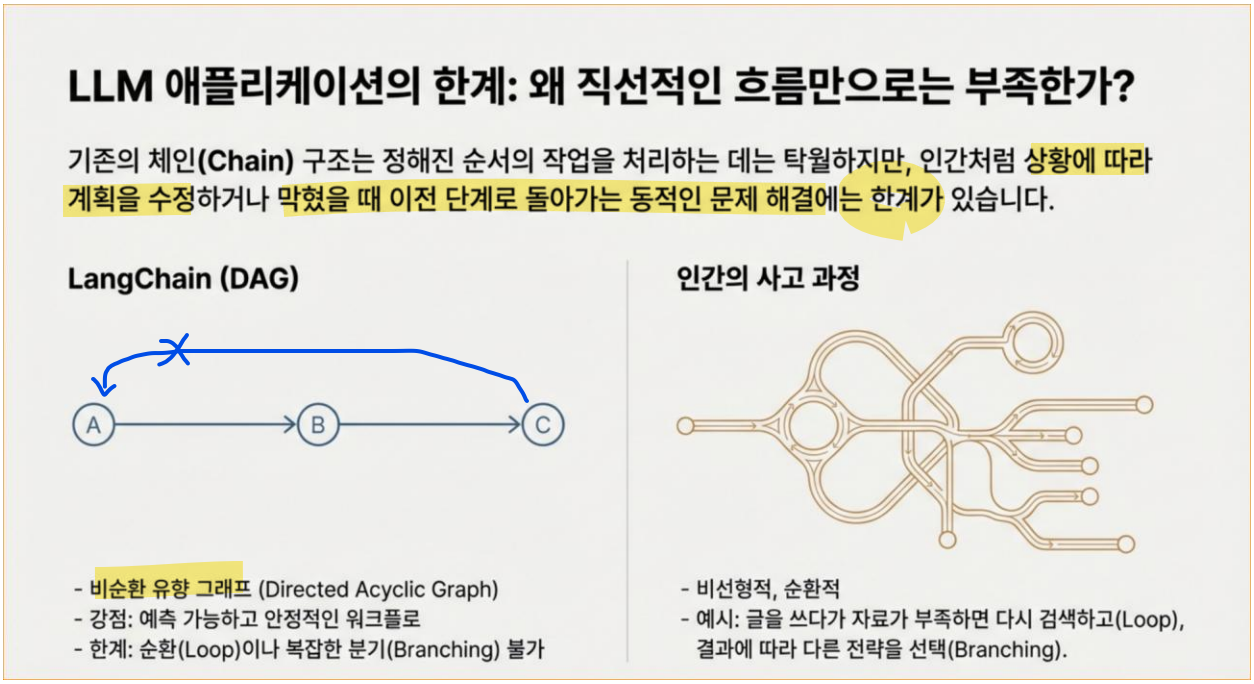
## Chapter 12

# 랭그래프(LangGraph)를 활용한 자율 에이전트 구축

## 제 1 부: 에이전트 시스템의 패러다임 전환과 랭그래프의 기초

### 1. 서론: 왜 랭그래프인가?

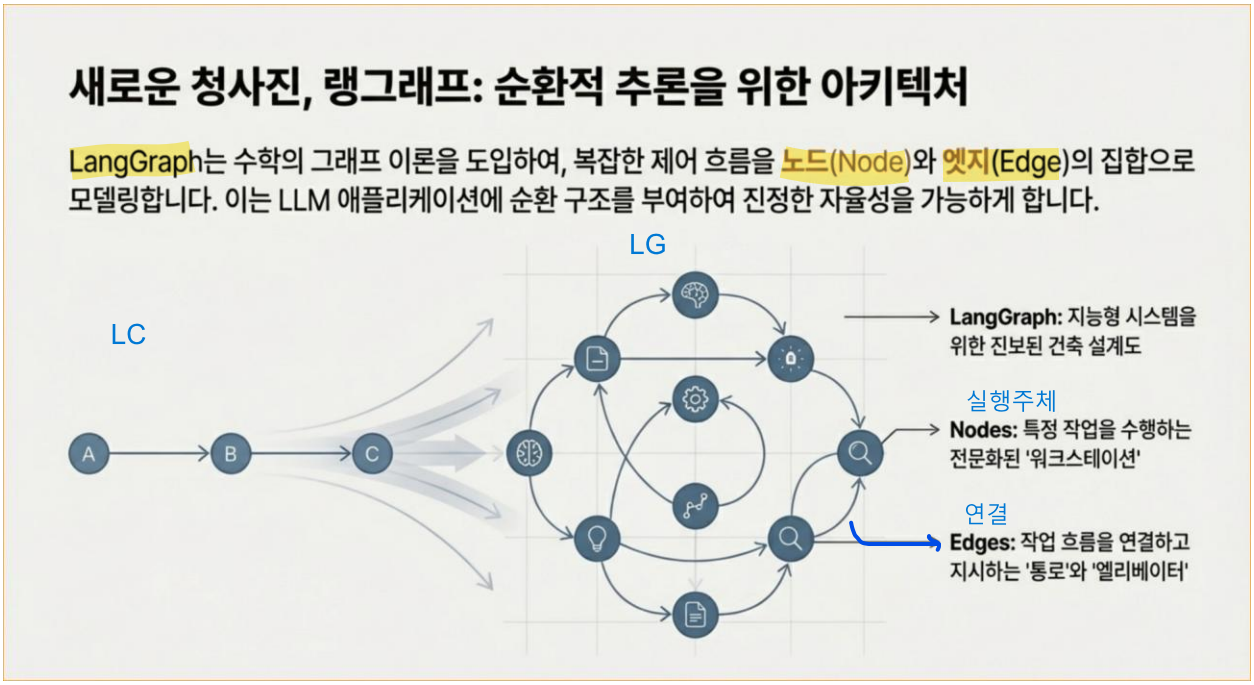



인공지능 애플리케이션 개발의 역사는 대규모 언어 모델(LLM)의 등장과 함께 급격한 진화를 거듭해왔습니다. 초기 LLM 애플리케이션이 사용자의 질문에 단답형으로 응답하는 단순한 '질의응답(Q&A)' 시스템에 머물렀다면, 현재는 복잡한 비즈니스 로직을 수행하고, 외부 도구와 상호작용하며, 스스로 계획을 수립하고 수정하는 '에이전트(Agent)' 시스템으로 발전하고 있습니다. 이러한 변화의 중심에는 선형적인 작업 흐름을 넘어선 순환적(Cyclic)이고 동적인 제어 흐름에 대한 요구가 자리 잡고 있습니다.

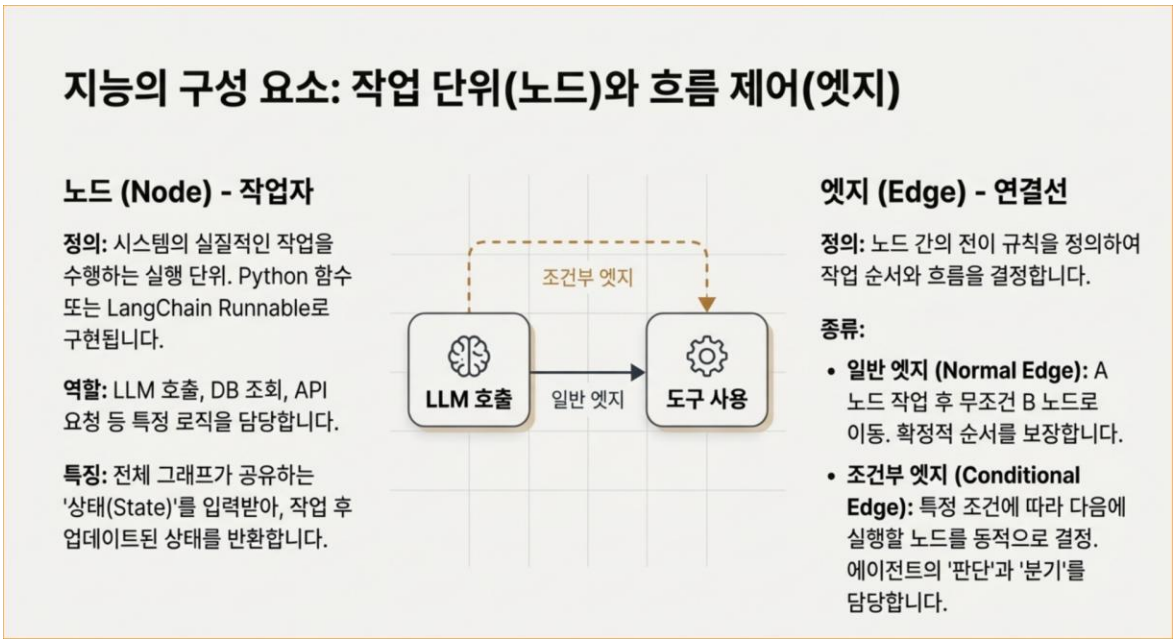
기존의 랭체인(LangChain) 프레임워크는 일련의 작업이 순차적으로 실행되는 체인(Chain) 구조, 즉 비순환 유향 그래프(DAG, Directed Acyclic Graph)를 구성하는 데 탁월한 성능을 발휘했습니다. 그러나 실제 세계의 문제 해결 과정은 직선적이지 않습니다. 인간은 문제를 해결하다가 막히면 다시 이전 단계로 돌아가고(Loop), 상황에 따라 다른 도구를 선택하며(Branching), 작업의 결과를

기억하고 이를 바탕으로 다음 행동을 결정합니다. 이러한 인간의 인지 과정을 모방하기 위해 등장한 프레임워크가 바로 랭그래프(LangGraph)입니다.

## 2. 그래프 이론에 기반한 아키텍처 설계




랭그래프는 이름에서 알 수 있듯이 수학적 '그래프 이론(Graph Theory)'을 소프트웨어 아키텍처에 도입했습니다. 이는 애플리케이션의 흐름을 노드(Node)와 엣지(Edge)의 집합으로 모델링함으로써, 복잡한 비즈니스 로직을 시각적이고 논리적인 구조로 환원시킵니다.




## 2.1 노드(Node): 지능적 작업의 실행 단위

랭그래프에서 노드는 시스템의 실질적인 '작업자'를 의미합니다. 기술적으로 노드는 파이썬 함수(Function) 또는 랭체인의 러너블(Runnable) 객체로 구현됩니다. 각 노드는 그래프의 현재 상태(State)를 입력으로 받아, 정의된 로직(LLM 호출, DB 조회, API 요청 등)을 수행한 후, 그 결과로 업데이트된 상태를 반환합니다.

전통적인 프로그래밍의 함수와 랭그래프 노드의 결정적인 차이는 '상태의 공유'에 있습니다. 일반적인 함수는 인자로 전달받은 데이터만 처리하지만, 랭그래프의 노드는 전체 그래프가 공유하는 전역적인 상태 스키마에 접근하고 이를 수정할 수 있는 권한을 가집니다. 이는 마치 회사에서 각 부서(노드)가 중앙 데이터베이스(상태)를 조회하고 자신의 업무 결과를 기록하는 것과 유사합니다.

### 2.2 엣지(Edge): 제어 흐름과 의사결정의 시각화

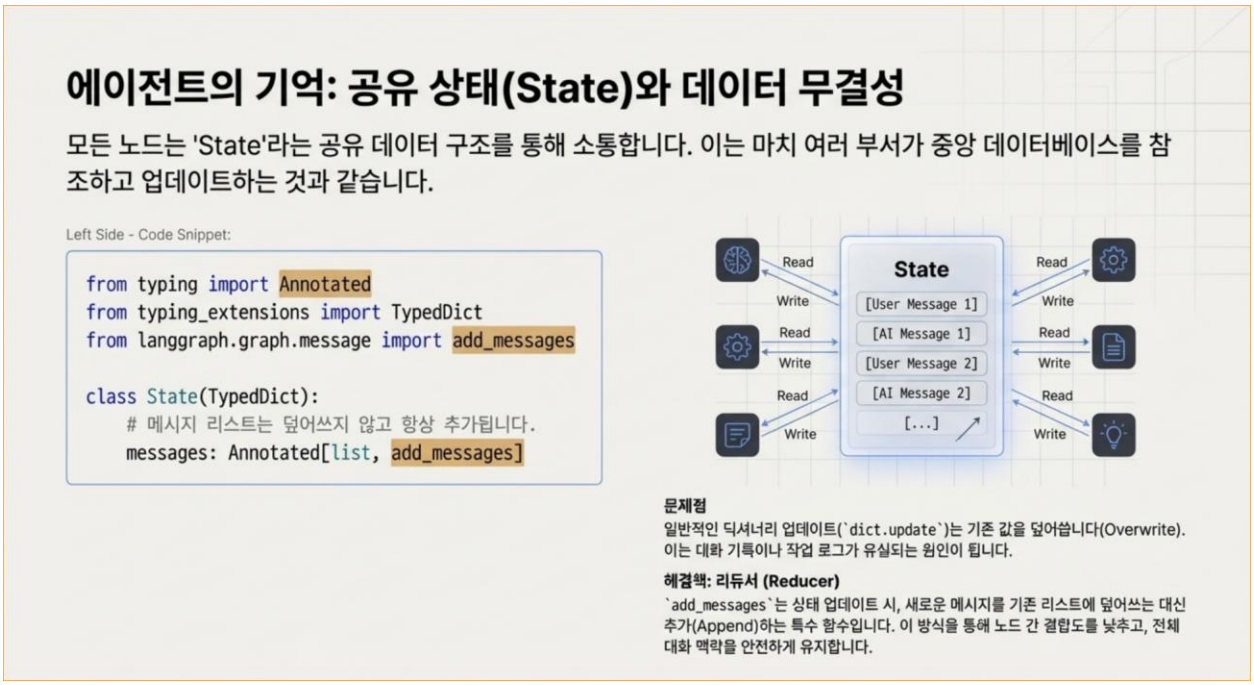
엣지는 노드와 노드 사이의 연결선으로, 작업의 전이(Transition) 규칙을 정의합니다. 랭그래프는 두 가지 형태의 엣지를 통해 유연한 워크플로를 지원합니다.

1. **일반 엣지 (Normal Edge)**: 가장 기본적인 형태로, 선행 노드의 작업이 끝나면 무조건 후행 노드로 제어권을 넘깁니다. 이는 프로세스의 확정적인 순서를 보장합니다.
2. **조건부 엣지 (Conditional Edge)**: 에이전트 시스템의 핵심인 '판단'과 '분기'를 담당합니다.

특정 노드의 실행 결과나 현재 상태 값을 기반으로, 다음에 실행할 노드를 동적으로 결정합니다. 예를 들어, "검색 결과가 충분한가?"라는 질문에 대해 "예"라면 기사 작성 노드로, "아니오"라면 재검색 노드로 분기하는 로직이 이에 해당합니다.[6]

이러한 조건부 엣지는 랭그래프가 단순한 파이프라인 도구가 아니라, 상황에 따라 스스로 경로를 수정하는 자율 에이전트 프레임워크임을 증명하는 요소입니다.

## 3. 상태(State) 관리 시스템: 에이전트의 기억




### 3.1 상태 스키마(Schema)의 정의

랭그래프의 모든 노드는 State 라고 불리는 공유 데이터 구조를 통해 소통합니다. 파이썬의 TypedDict 를 사용하여 이 상태를 정의하고 있습니다.

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class State(TypedDict):
    # 메시지 리스트를 저장하며, add_messages 리듀서를 사용
    messages: Annotated[list[str], add_messages]
```

여기서 주목해야 할 점은 Annotated 와 add_messages 의 사용입니다. 일반적인 딕셔너리 업데이트(dict.update)는 키 값이 같을 경우 기존 값을 덮어쓰기(Overwrite)합니다. 그러나 대화형

시스템이나 에이전트 로그에서는 과거의 데이터를 보존하면서 새로운 데이터를 누적(Append)해야 하는 경우가 빈번합니다.

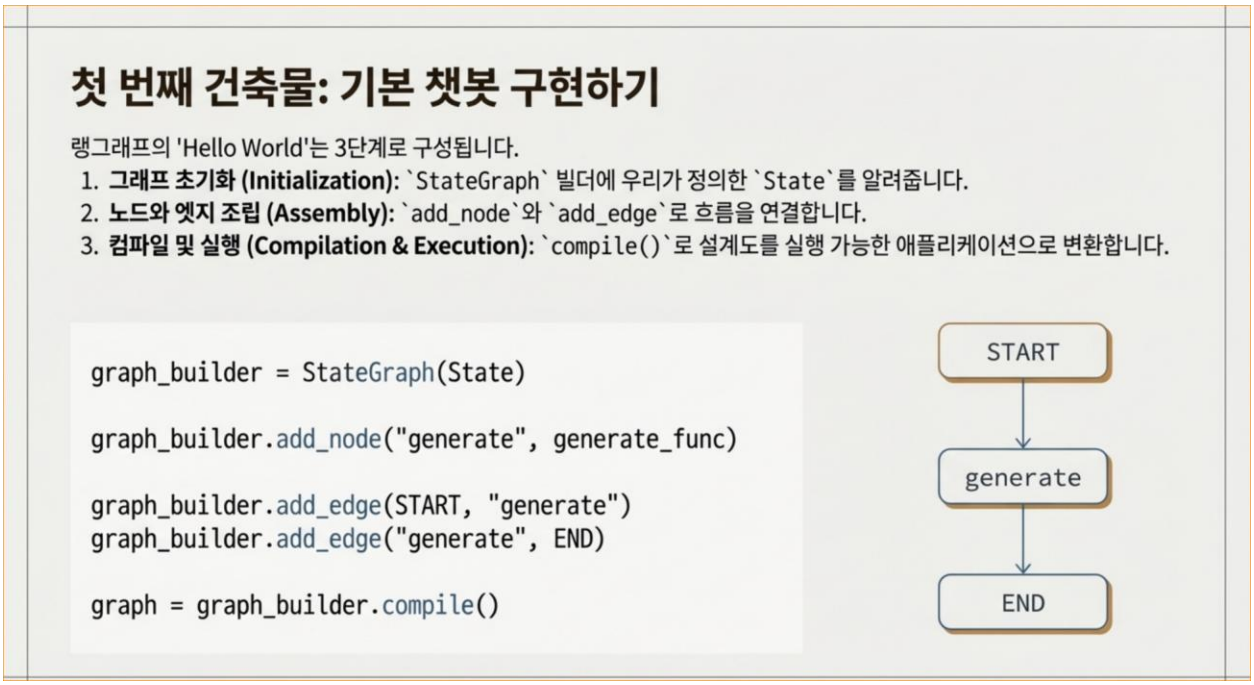
### 3.2 리듀서(Reducer) 패턴의 이해

add_messages 는 랭그래프가 제공하는 강력한 리듀서 함수입니다. 리듀서는 상태 업데이트 요청이 들어왔을 때, 이를 기존 상태와 어떻게 병합할지를 결정하는 로직을 담고 있습니다.

- **작동 메커니즘**: 노드가 {"messages": [new_msg]}를 반환하면, add_messages 리듀서는 이를 받아 기존의 current_state["messages"] 리스트 뒤에 new_msg 를 이어 붙입니다.
- **데이터 무결성**: 이 방식은 각 노드가 전체 상태를 알 필요 없이, 자신이 생성한 데이터만 반환하면 되도록 하여 노드 간의 결합도(Coupling)를 낮추고 독립성을 높입니다. 또한, HumanMessage, AIMessage, ToolMessage 등 다양한 메시지 타입을 자동으로 식별하고 관리하여 대화의 맥락이 끊기지 않도록 보장합니다.

# 제 2 부: 실전 구축 - 기본 챗봇에서 메모리 시스템까지

## 4. 기본 챗봇 구현




### 4.1 모델 및 그래프 초기화

우선적으로 LLM 모델(OpenAI GPT)을 초기화하고, 앞서 정의한 State 클래스를 기반으로 StateGraph 빌더를 생성합니다.

```
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END

# 모델 설정 (GPT-4o-mini 활용)
model = ChatOpenAI(model="gpt-4o-mini")

# 그래프 빌더 생성
graph_builder = StateGraph(State)
```

StateGraph(State) 선언은 이 그래프가 State 타입의 데이터를 처리하는 전용 기계임을 명시하는 것입니다. 이는 정적 타입 검사와 런타임 데이터 검증을 가능하게 하여, 복잡한 에이전트 시스템에서 발생할 수 있는 데이터 구조 오류를 미연에 방지합니다.

### 4.2 노드와 엣지의 조립

다음으로 챗봇의 핵심 로직인 generate 함수를 정의하고 그래프에 추가합니다.

```
def generate(state: State):
    # 현재까지의 대화 기록(state["messages"])을 모델에 전달
    return {"messages": [model.invoke(state["messages"])]}

# 노드 등록
graph_builder.add_node("generate", generate)

# 흐름 연결: 시작 -> 생성 -> 종료
graph_builder.add_edge(START, "generate")
graph_builder.add_edge("generate", END)
```

여기서 START 노드는 그래프 실행 시 최초로 진입하는 가상의 노드이며, END 노드는 워크플로의 종료를 의미합니다. generate 노드는 START 에서 받은 사용자 입력을 처리하여 응답을 생성하고, 그 직후 END 로 이동하여 실행을 마칩니다. 이는 가장 단순한 형태의 선형 그래프 구조입니다.

### 4.3 그래프 컴파일(Compile)과 실행

설계된 그래프는 compile() 메서드를 통해 실행 가능한 애플리케이션(CompiledGraph)으로 변환됩니다. 컴파일 과정에서는 고아 노드(Orphaned Node, 연결되지 않은 노드)가 없는지, 상태 스키마가 일치하는지 등의 유효성 검사가 수행됩니다.

```
graph = graph_builder.compile()
```

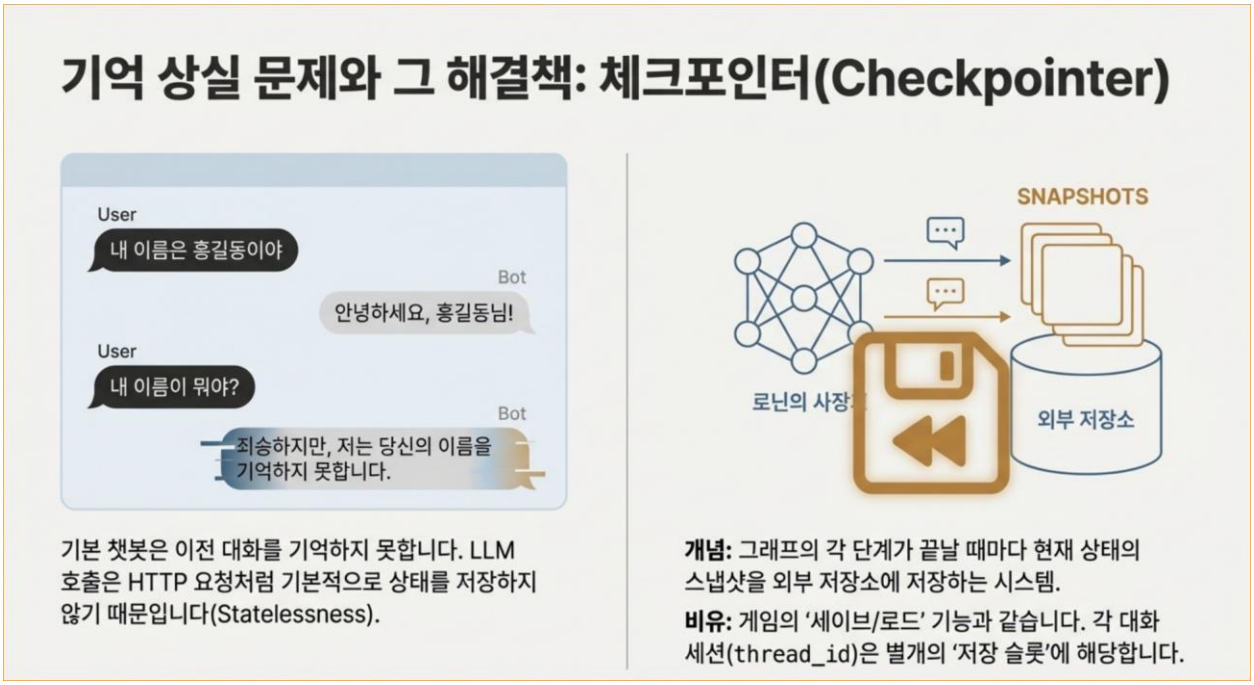
실행은 invoke 또는 stream 메서드를 사용합니다. 특히 챗봇 애플리케이션에서는 사용자 경험을 위해 스트리밍이 필수적입니다.

```
# 스트리밍 실행 예시
for chunk in graph.stream(input_data, stream_mode="messages"):
    print(chunk.content, end="")
```

stream_mode="messages" 옵션은 LLM 이 생성하는 토큰 단위의 출력을 실시간으로 반환합니다. 이는 사용자가 AI 의 응답을 기다리는 체감 시간을 줄여줍니다.

## 5. 메모리 시스템과 영속성

기본 챗봇의 한계는 '기억 상실'입니다. HTTP 요청과 마찬가지로 LLM 호출은 기본적으로 상태를 저장하지 않기 때문에, 이전 대화의 맥락을 유지하기 위해서는 별도의 저장 메커니즘이 필요합니다. 랭그래프의 **체크포인터(Checkpointer)** 시스템을 통해 이를 해결합니다.




## 5.1 체크포인터의 작동 원리

랭그래프의 체크포인터는 그래프의 각 '슈퍼 스텝(Super-step)'이 완료될 때마다 현재의 상태 스냅샷을 저장소에 직렬화(Serialize)하여 저장합니다.

| 컴포넌트 | 역할 | 비유 |
|---|---|---|
| Checkpointer | 상태 저장 및 로드 담당 | 게임의 '세이브/로드' 시스템 |
| Thread ID | 특정 대화 세션 식별자 | 저장된 게임 슬롯 번호 |
| Checkpoint | 특정 시점의 전체 상태 데이터 | 특정 시점의 세이브 파일 |

## 5.2 MemorySaver 구현과 한계

개발용 인메모리 저장소인 MemorySaver 를 사용합니다.

```
from langgraph.checkpoint.memory import MemorySaver

# 체크포인터 인스턴스 생성
memory = MemorySaver()

# 컴파일 시 체크포인터 주입
graph = graph_builder.compile(checkpointer=memory)

# 실행 시 스레드 ID 지정
config = {"configurable": {"thread_id": "abcd"}}
events = graph.stream(input_data, config=config)
```

이 코드가 실행되면 랭그래프는 다음 절차를 따릅니다:

1. config 에서 thread_id("abcd")를 추출합니다.
2. memory 저장소에서 "abcd" ID 에 해당하는 마지막 체크포인트가 있는지 확인합니다.
3. 있다면 그 상태(이전 대화 목록)를 불러와 현재 입력 메시지를 덧붙입니다(add_messages).
4. 없다면 새로운 상태를 초기화합니다.
5. 그래프를 실행하고, 실행 완료 후 최신 상태를 다시 "abcd" ID 로 저장합니다.[1]

**[프로덕션 환경에서의 고려사항]**

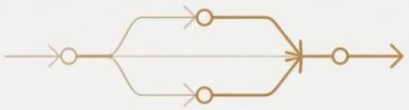
MemorySaver 는 RAM 에 데이터를 저장하므로 프로세스가 재시작되면 데이터가 소실됩니다. 실제 서비스 환경(Production)에서는 PostgresSaver 나 RedisSaver 와 같은 영구 저장소 기반의 체크포인터를 사용해야 합니다. 이는 대용량 트래픽 처리 시 데이터의 내구성을 보장하고, 여러 서버 인스턴스 간에 상태를 공유할 수 있게 해줍니다.

## 체크포인터의 작동 원리: 세션 영속성과 프로덕션 환경

```
graph = builder.compile(checkpointer=memory)
```

1. 실행 시 'config={"configurable": {"thread_id": "user_123"}}를 통해 대화 세션을 식별합니다.
2. 체크포인터는 저장소에서 'user_123' ID의 마지막 상태를 불러옵니다.
3. 그래프 실행 후, 업데이트된 최신 상태를 다시 'user_123' ID로 저장합니다.

| 특징 | MemorySaver | PostgresSaver | RedisSaver |
|---|---|---|---|
| 저장 위치 | 로컬 메모리 (RAM) | PostgreSQL DB | Redis 인메모리 DB |
| 영속성 | 프로세스 종료 시 삭제 | 영구 보존 | 설정에 따라 영구 보존 |
| 용도 | 개발, 테스트, 데모 | 실제 서비스, 장기 기억 | 고성능 캐싱, 세션 관리 |



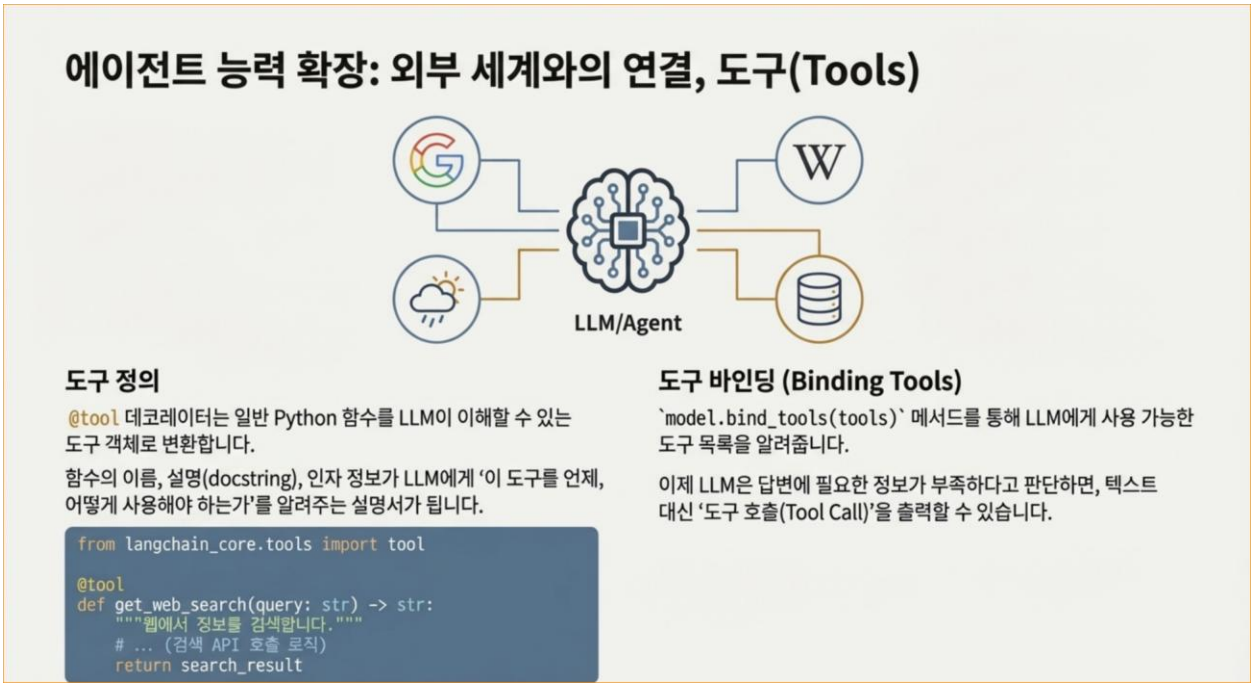
**고급 기능:** 상태가 버전별로 저장되므로, 과거 시점으로 돌아가는 '시간 여행(Time Travel)' 디버깅이나 '인간 개입(Human-in-the-loop)' 워크플로 구현의 기반이 됩니다.

### 5.3 시간 여행(Time Travel)과 휴먼 인 더 루프(Human-in-the-Loop)

체크포인팅 기능은 단순한 대화 기억을 넘어 강력한 디버깅 및 제어 기능을 제공합니다. 상태가 버전별로 저장되므로, 개발자는 과거의 특정 시점으로 돌아가서(Time Travel) 다른 입력을 주거나 로직을 수정하여 재실행할 수 있습니다. 또한, 사람이 중간에 개입하여 AI 의 결정을 승인하거나 수정하는 'Human-in-the-loop' 워크플로를 구현하는 기반이 됩니다.

# 제 3 부: 고급 에이전트 - 도구 사용과 자율적 의사결정

## 6. 도구(Tools) 통합과 실행




랭그래프의 정점인 '도구를 사용하는 에이전트'를 구현합니다. 시나리오는 "인터넷 검색을 통해 최신 정보를 수집하고 기사를 작성하는 기자 봇"입니다.

### 6.1 도구의 정의와 바인딩

에이전트가 외부 세계와 상호작용하기 위해서는 도구가 필요합니다. 문서에서는 get_current_time 과 get_web_search 라는 두 가지 도구를 정의합니다.

```
from langchain_core.tools import tool

@tool
def get_web_search(query: str, search_period: str='m') -> str:
    """웹 검색을 수행하는 함수..."""
    # DuckDuckGo API 호출 로직
    return search_result
```

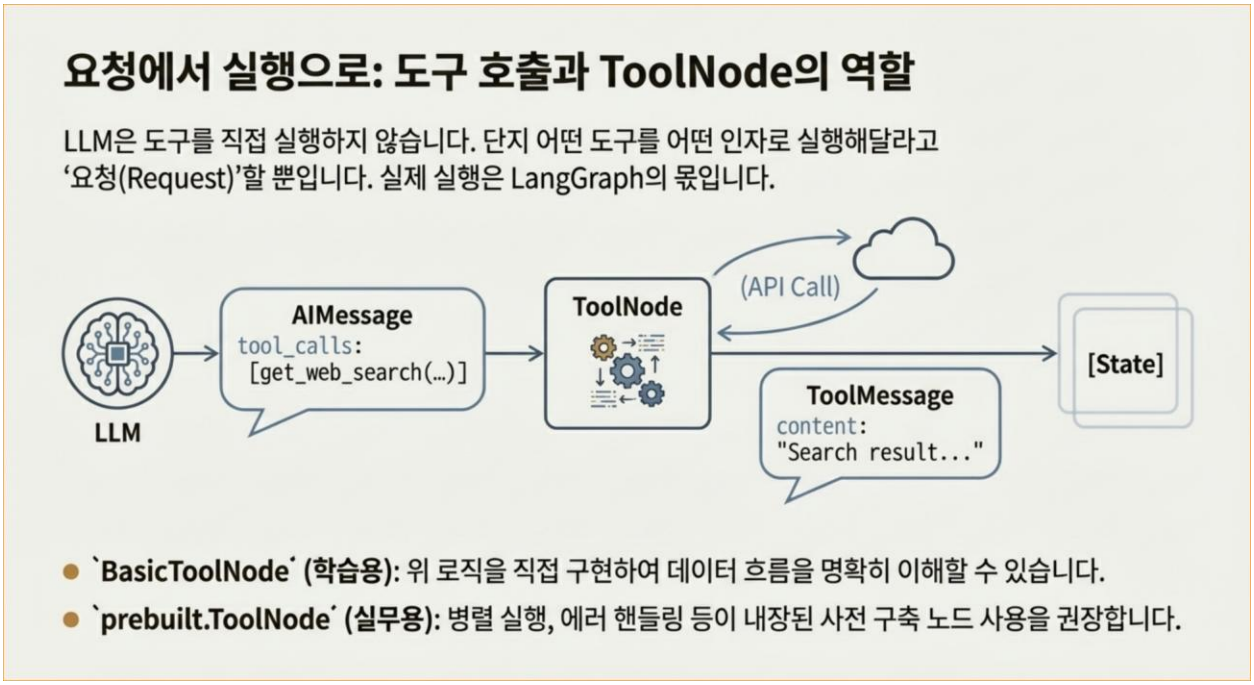
@tool 데코레이터는 파이썬 함수를 랭체인 도구 객체로 변환하며, 함수의 이름, 설명(docstring), 인자 타입 정보를 추출하여 스키마를 생성합니다. 이 스키마는 LLM 에게 "이 도구는 언제, 어떻게 사용해야 하는가"를 알려주는 설명서 역할을 합니다.

```
tools = [get_current_time, get_web_search]
model_with_tools = model.bind_tools(tools)
```

bind_tools 메서드는 LLM 에게 이 도구들의 존재를 알립니다. 이제 모델은 사용자의 질문에 답변하기 위해 지식이 부족하다고 판단되면, 텍스트를 생성하는 대신 '도구 호출(Tool Call)'이라는 특별한 구조의 메시지를 출력할 수 있습니다.

### 6.2 도구 실행 노드 (BasicToolNode vs ToolNode)

LLM 은 도구를 '실행'하지 않습니다. 단지 '실행해달라'고 요청(Request)할 뿐입니다. 실제 도구 함수를 실행하고 그 결과를 받아오는 것은 랭그래프의 몫입니다. 이를 위해 BasicToolNode 를 직접 구현합니다.




```
class BasicToolNode:
    def __init__(self, tools: list) -> None:
        self.tools_by_name = {tool.name: tool for tool in tools}

    def __call__(self, inputs: dict):
        # 최신 메시지에서 도구 호출 요청 확인
        message = inputs["messages"][-1]
        outputs =
        for tool_call in message.tool_calls:
            # 해당 도구 함수 실행
            tool_result = self.tools_by_name[tool_call["name"]].invoke(tool_call["args"])
```

```
        # 결과를 ToolMessage 로 포장
        outputs.append(ToolMessage(
            content=json.dumps(tool_result),
            name=tool_call["name"],
            tool_call_id=tool_call["id"]
        ))
    return {"messages": outputs}
```

---

상태에서 AIMessage 를 꺼내 tool_calls 속성을 파싱하고, 매칭되는 도구를 실행한 뒤, 그 결과를 ToolMessage 형태로 다시 상태에 추가합니다.

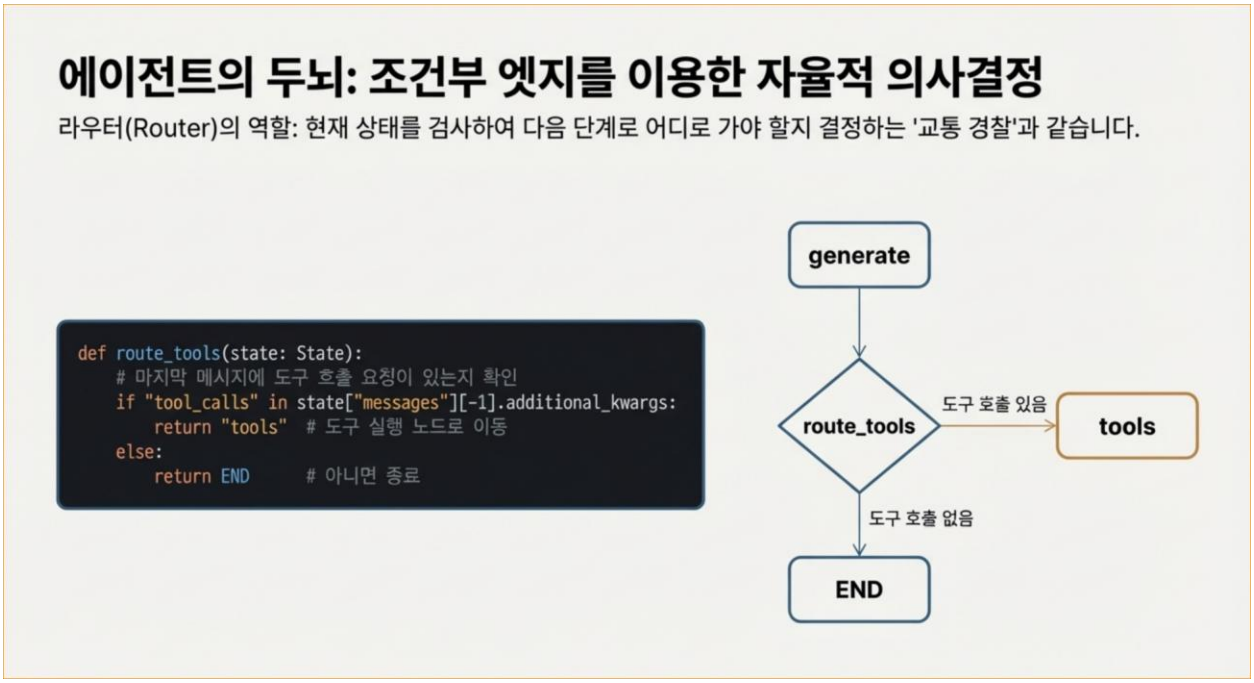
---

최신 랭그래프 라이브러리는 prebuilt.ToolNode 를 제공합니다. 실무에서는 직접 구현하기보다 이 사전 구축된 노드를 사용하는 것이 권장됩니다. prebuilt.ToolNode 는 병렬 실행 처리, 에러 핸들링, InjectedState(도구 내부에서 그래프 상태 접근) 등 더 고도화된 기능을 내장하고 있기 때문입니다. 하지만 학습 단계에서는 BasicToolNode 를 통해 데이터의 흐름을 직접 제어해보는 경험이 중요합니다.

---

## 7. 라우팅(Routing)과 순환적 추론

에이전트가 진정한 자율성을 가지려면 상황에 따라 경로를 선택해야 합니다. 이를 위해 조건부 엣지(Conditional Edge)와 **라우터(Router)** 패턴이 사용됩니다.




### 7.1 라우터 로직 (route_tools)

라우터 함수는 현재 상태를 검사하여 다음 목적지(노드 이름)를 반환하는 '교통 경찰'과 같습니다.

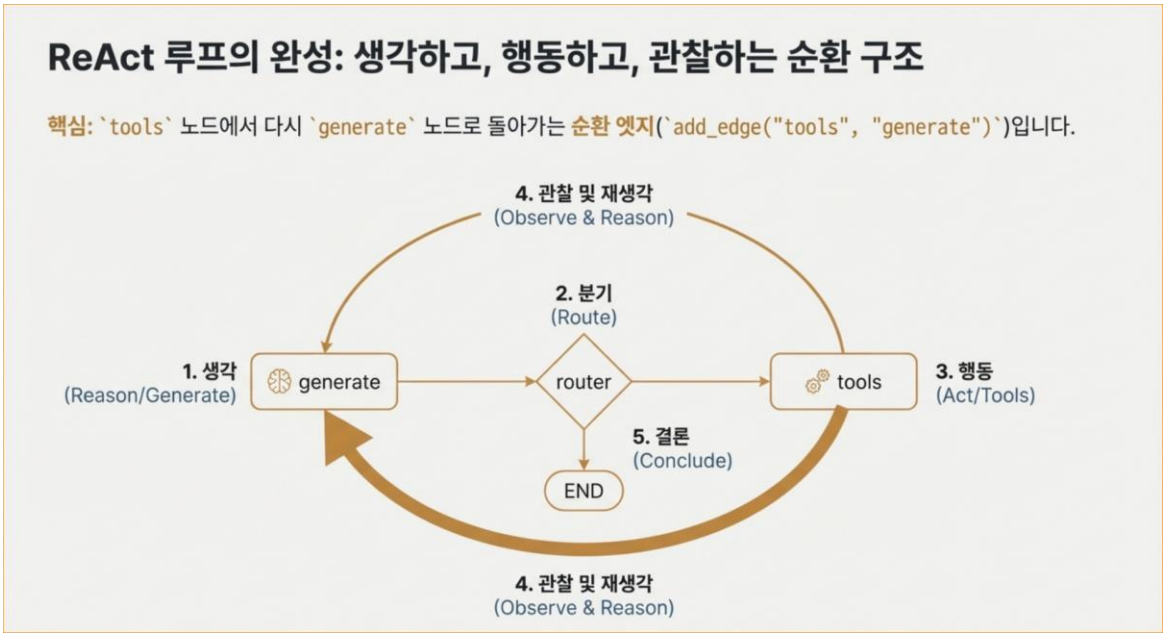
```
def route_tools(state: State):
    ai_message = state["messages"][-1]
    # AI 가 도구 호출을 요청했는지 확인
    if hasattr(ai_message, "tool_calls") and len(ai_message.tool_calls) > 0:
        return "tools"  # 도구 실행 노드로 이동
    return END          # 아니면 종료
```

이 함수는 매우 직관적입니다. LLM 이 도구를 쓰고 싶어 하면 도구 노드로 보내고, 아니면(즉, 최종 답변을 생성했으면) 작업을 마칩니다.

## 7.2 그래프 구성과 순환(Cycle)

```
# 조건부 엣지 추가
graph_builder.add_conditional_edges(
    "generate",       # 출발 노드
    route_tools,      # 결정 함수
    {"tools": "tools", END: END} # 경로 매핑
)
# 순환 엣지 추가: 도구 실행 후 다시 생성 노드로 복귀
graph_builder.add_edge("tools", "generate")
```

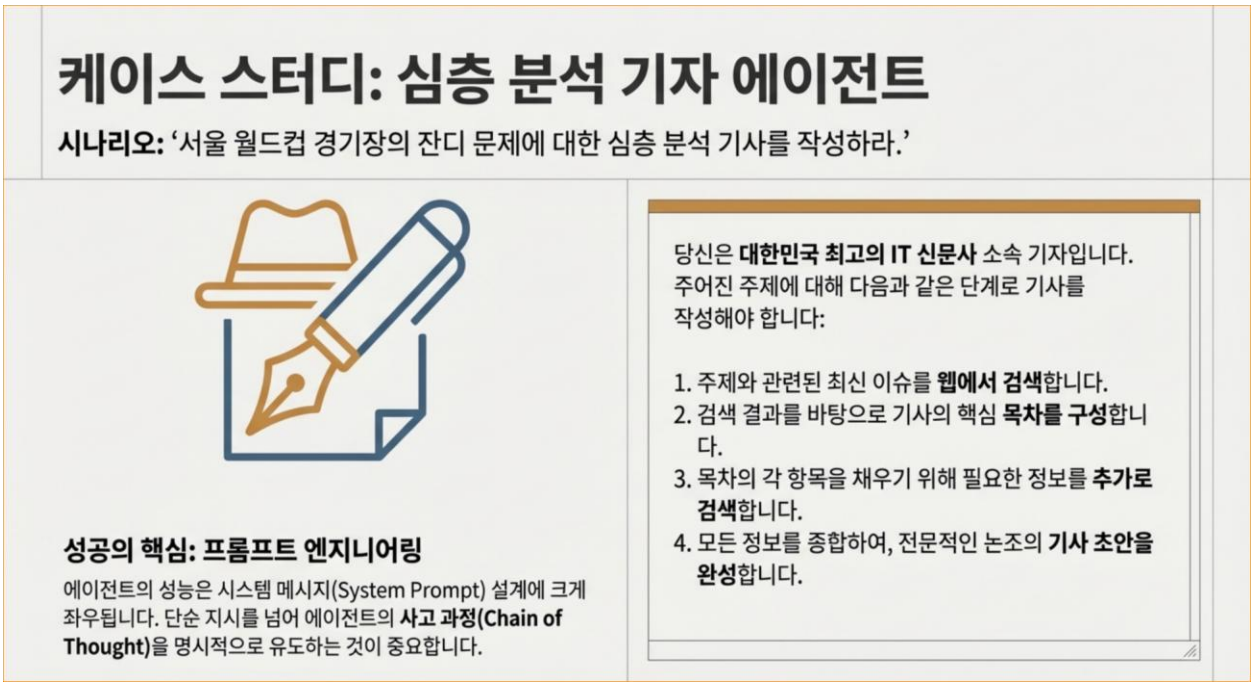
가장 중요한 부분은 tools 노드에서 다시 generate 노드로 돌아가는 **순환 엣지**입니다. 이 연결이 있기에 에이전트는 다음과 같은 'ReAct(Reasoning + Acting)' 루프를 수행할 수 있습니다




1. **생각(Generate)**: "A 에 대한 정보가 필요해. 검색하자."
2. **행동(Tools)**: (검색 수행 후 결과 반환)
3. **관찰/재생각(Generate)**: "검색 결과를 보니 B 에 대한 정보가 빠졌네. 다시 검색하자."
4. **행동(Tools)**: (추가 검색 수행)
5. **결론(Generate)**: "이제 정보가 충분하니 기사를 작성하자." -> **종료(END)**

이러한 순환 구조는 에이전트가 한 번의 시도로 끝내지 않고, 목표를 달성할 때까지 끈질기게 시도하고 수정하는 능력을 부여합니다.

# 제 4 부: 케이스 스터디 - 심층 분석 기자 에이전트




## 8. 시나리오 실행 및 분석

"서울 월드컵 경기장 잔디 문제" 기사 작성 예제는 앞서 배운 모든 개념의 집약체입니다.

### 8.1 프롬프트 엔지니어링: 페르소나 부여

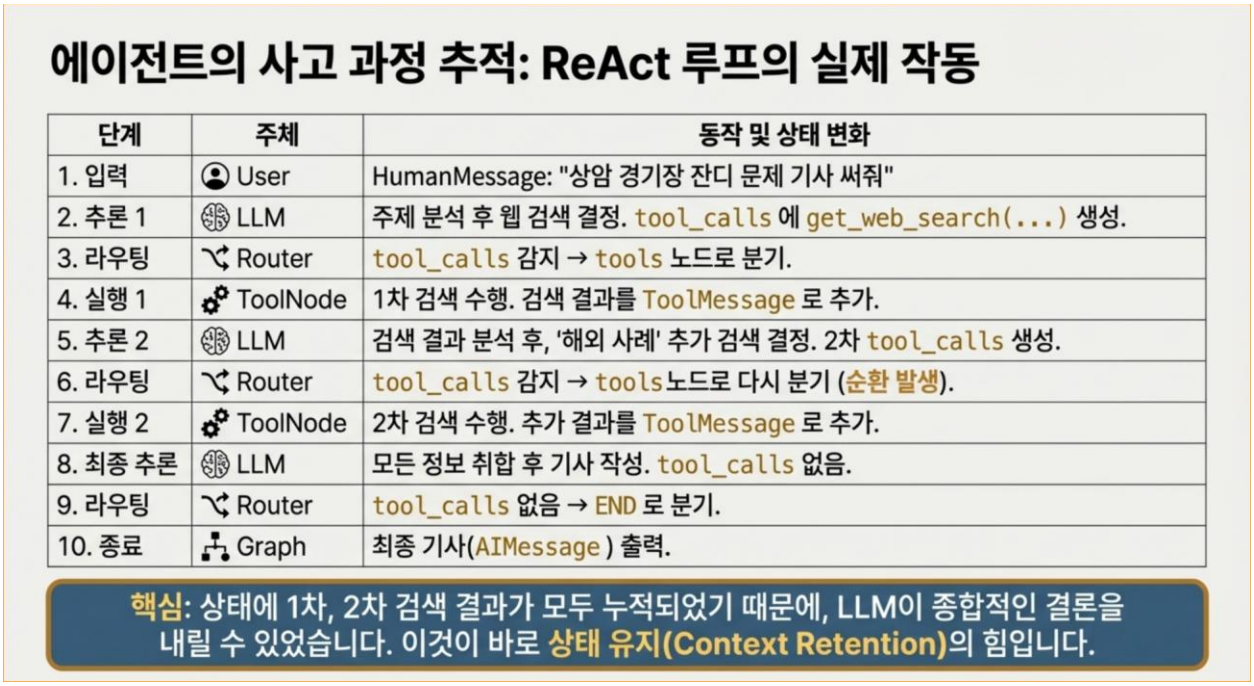
에이전트의 성능은 시스템 메시지(System Prompt) 설계에 크게 좌우됩니다.

> "너는 신문기자이다... 최근 이슈를 검색하고... 목차를 만들고... 추가 검색을 통해 내용을 채우고... 신문사 신문 형식으로 기사를 작성하라."

이 프롬프트는 단순한 지시가 아니라 에이전트의 **사고 과정(Chain of Thought)**을 강제합니다. "검색 -> 선정 이유 설명 -> 목차 작성 -> 재검색 -> 기사 작성"이라는 명시적인 단계 지시는 LLM 이 복잡한 작업을 작은 단위로 쪼개어 처리하게 돕습니다.

### 8.2 실행 추적 (Trace Analysis)

실제 실행 로그를 분석해보면 에이전트의 사고 흐름을 명확히 볼 수 있습니다.

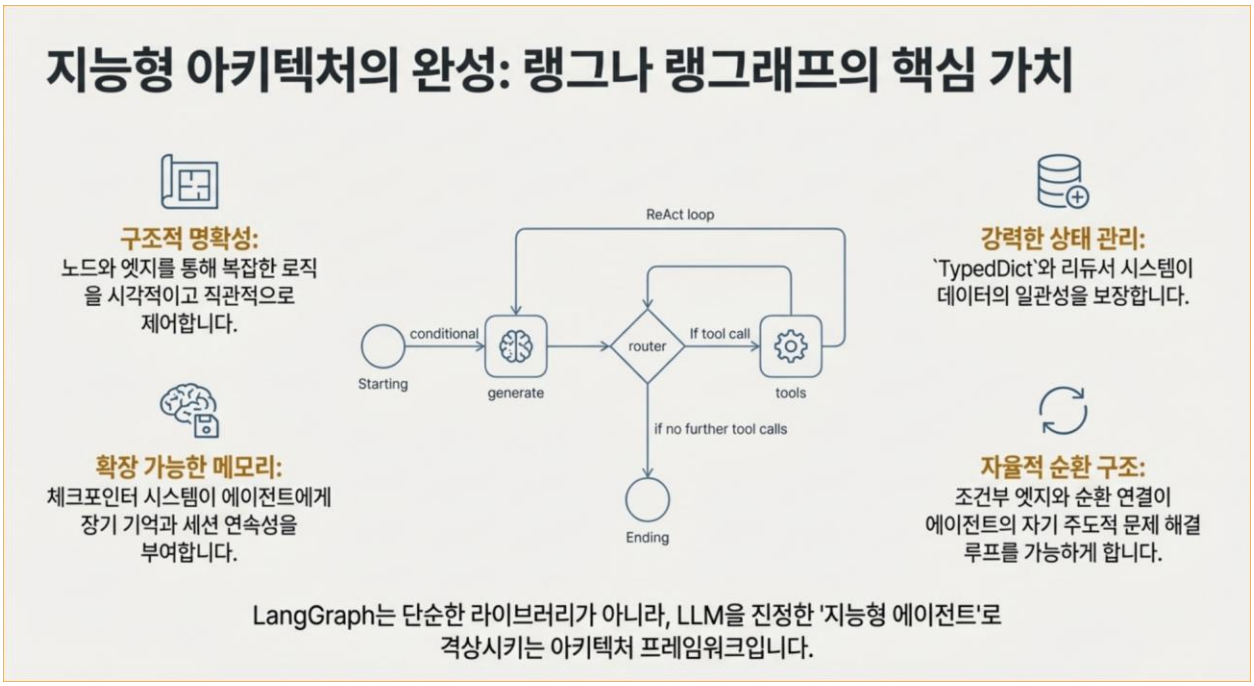


## 에이전트의 사고 과정 추적: ReAct 루프의 실제 작동

| 단계 | 주체 | 동작 및 상태 변화 |
|---|---|---|
| 1. 입력 | User | HumanMessage: "상암 경기장 잔디 문제 기사 써줘" |
| 2. 추론 1 | LLM | 주제 분석 후 웹 검색 결정. `tool_calls` 에 `get_web_search(...)` 생성. |
| 3. 라우팅 | Router | `tool_calls` 감지 → `tools` 노드로 분기. |
| 4. 실행 1 | ToolNode | 1차 검색 수행. 검색 결과를 `ToolMessage` 로 추가. |
| 5. 추론 2 | LLM | 검색 결과 분석 후, '해외 사례' 추가 검색 결정. 2차 `tool_calls` 생성. |
| 6. 라우팅 | Router | `tool_calls` 감지 → `tools` 노드로 다시 분기 (**순환 발생**). |
| 7. 실행 2 | ToolNode | 2차 검색 수행. 추가 결과를 `ToolMessage` 로 추가. |
| 8. 최종 추론 | LLM | 모든 정보 취합 후 기사 작성. `tool_calls` 없음. |
| 9. 라우팅 | Router | `tool_calls` 없음 → `END` 로 분기. |
| 10. 종료 | Graph | 최종 기사(`AIMessage` ) 출력. |

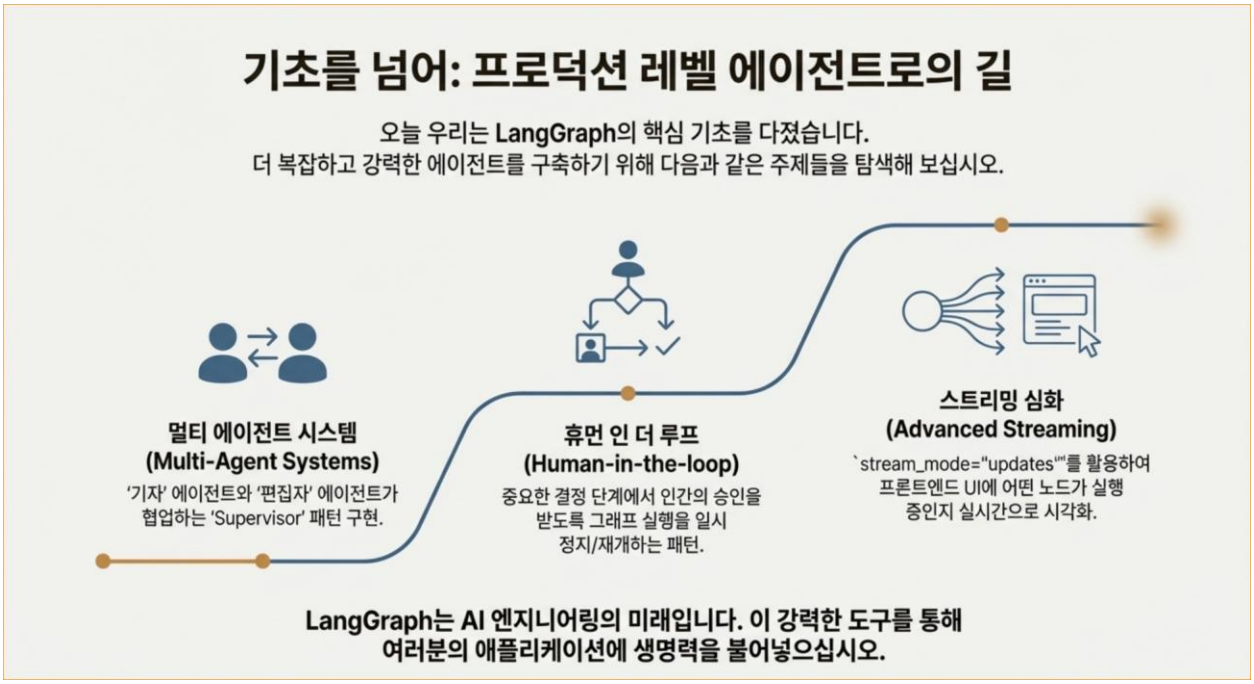
**핵심**: 상태에 1차, 2차 검색 결과가 모두 누적되었기 때문에, LLM이 종합적인 결론을 내릴 수 있었습니다. 이것이 바로 **상태 유지(Context Retention)**의 힘입니다.

### 8.3 결과물의 시사점




생성된 기사는 단순한 텍스트 생성이 아닙니다. 검색된 팩트("서울시 예산 33 억 증액", "해외 사례 연구 부재" 등)를 근거로 논리적인 글을 완성했습니다. 이는 랭그래프의 **상태 유지(Context**

**Retention)** 기능이 없었다면 불가능했습니다. 1 차 검색 결과와 2 차 검색 결과가 모두 messages 상태 리스트에 누적되어 있었기에, LLM 은 이 모든 정보를 종합하여 최종 결론을 내릴 수 있었습니다.




**부록: 핵심 코드 치트 시트 (Cheat Sheet)**

| 기능 | 코드 패턴 | 설명 |
|---|---|---|
| 상태 정의 | messages: Annotated[list, add_messages] | 메시지 자동 병합(Append) 리듀서 설정 |
| 그래프 빌드 | builder = StateGraph(State) | 상태 스키마 기반 그래프 초기화 |
| 조건부 엣지 | builder.add_conditional_edges(...) | 동적 경로 분기 (Router) 설정 |
| 메모리 설정 | builder.compile(checkpointer=memory) | 체크포인터 주입을 통한 영속성 확보 |
| 실행(쓰레드) | config={"configurable": {"thread_id": "1"}} | 특정 세션 ID 로 상태 로드 및 실행 |
| 도구 바인딩 | llm.bind_tools(tools) | LLM 에게 사용 가능한 도구 인지시키기 |